

深圳大学实验报告

课程名称： 互联网编程

实验项目名称： 传输协议与套接字应用编程

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 李梦柯

报告人： 姚泽棉 学号： 2024150026 班级： 高性能班

实验时间： 2026/06/04——2026/06/25

实验报告提交时间： 2026/06/15

教务处制

一、实验目的与内容：

目的：

1. 掌握服务器端 Socket 编程开发能力。
2. 掌握 HTTP 协议并能用于实际编程开发。
3. 掌握 cookie 编程技术，会应用于在 web 应用开发中持久存储和传递客户端状态，会编程管理 cookie。

内容要求：

基于 java 编程实现一个 HTTP 服务器程序（20 分）和 HTTP 客户端程序（15 分），要求

1. 采用多线程技术或线程池编程技术处理客户端请求，支持多客户端同时访问；（10 分）
2. 实现 GET、HEAD 和 POST 请求，对客户端发送的不同请求给予正确响应；（15 分）
3. 在服务器上放一个静态网站（由 HTML 文本、图片文件或 JS 文件等组成），能根据不同请求，返回包括文本和图像 2 种（及以上）类型的响应，客户端可以正确显示和访问。（10 分）
4. HTTP 客户端程序能与该 HTTP 服务器连接并展示响应结果，正确发送不同类型的请求。（10 分）
5. 能使用 cookie 编程技术保存和传递会话状态信息，比如保存用户信息等，需要保存的信息可自行决定。（10 分）
6. 对服务器进行性能分析。对服务器进行压力测试，测试可支持多少个客户端同时访问，测试可支持多少个文件同时传输等。（10 分）

注意：

1. 实验报告中需要有实验结果的截屏图像。

二、实验过程和代码与结果

1. 给出满足内容要求的 HTTP 服务器端程序源码和 HTTP 客户端程序源码，简述设计思路或实验过程。

服务器端核心代码 SimpleHttpServer.java

```
ServerSocket server = new ServerSocket(8080);
ExecutorService pool = Executors.newFixedThreadPool(32);

while (true) {
    Socket socket = server.accept();
    pool.submit(() -> handle(socket));
}

void handle(Socket socket) {
    BufferedReader reader = new BufferedReader(
        new InputStreamReader(socket.getInputStream()));
```

```

OutputStream out = socket.getOutputStream();

String requestLine = reader.readLine();
String[] arr = requestLine.split(" ");
String method = arr[0];
String path = arr[1];

Map<String, String> headers = readHeaders(reader);

if ("POST".equals(method) && "/login".equals(path)) {
    int len = Integer.parseInt(headers.get("Content-Length"));
    char[] data = new char[len];
    reader.read(data);

    String username = new String(data).replace("username=", "");
    saveSession(headers.get("Cookie"), username);

    sendHtml(out, "<h1>POST 请求成功</h1><p>" + username + "</p>");
} else {
    if ("/".equals(path)) path = "/index.html";

    File file = new File("www" + path);
    byte[] body = Files.readAllBytes(file.toPath());

    if ("HEAD".equals(method)) {
        sendHeader(out, getType(file), body.length);
    } else {
        sendResponse(out, getType(file), body);
    }
}

socket.close();
}

```

说明：

服务器通过 `ServerSocket` 监听 8080 端口，使用线程池处理多个客户端连接。程序读取 HTTP 请求行，判断请求方法。如果是 `POST /login`，则读取请求体并保存用户名；如果是 `GET` 或 `HEAD`，则从 `www` 目录读取静态文件并返回。`HEAD` 请求只返回响应头，不返回响应体。

客户端核心代码 HttpClientDemo.java

```
String cookie = "";

void send(String method, String path, String body) throws Exception {
    Socket socket = new Socket("localhost", 8080);
    InputStream in = socket.getInputStream();
    OutputStream out = socket.getOutputStream();

    byte[] data = body.getBytes();

    String request =
        method + " " + path + " HTTP/1.1\r\n" +
        "Host: localhost:8080\r\n" +
        (cookie.isEmpty() ? "" : "Cookie: " + cookie + "\r\n") +
        ("POST".equals(method)
            ? "Content-Type: application/x-www-form-urlencoded\r\n" +
            "Content-Length: " + data.length + "\r\n"
            : "") +
        "\r\n";

    out.write(request.getBytes());
    if ("POST".equals(method)) {
        out.write(data);
    }

    BufferedReader reader = new BufferedReader(new InputStreamReader(in));
    String line;
    while ((line = reader.readLine()) != null) {
        if (line.startsWith("Set-Cookie:")) {
            cookie = line.substring(11).split(";")[0].trim();
        }
        System.out.println(line);
    }

    socket.close();
}

public static void main(String[] args) throws Exception {
    send("GET", "/", "");
    send("HEAD", "/images/network.svg", "");
    send("POST", "/login", "username=YaoZemian");
    send("GET", "/api/session", "");
}
```

说明：

客户端使用 Socket 连接服务器，手动拼接 HTTP 请求报文。程序依次发送 GET、HEAD、POST 和再次 GET 请求。收到服务器响应后，客户端打印响应内容，并保存 Set-Cookie 中的会话 ID，后续请求再通过 Cookie 请求头发送给服务器。

压力测试核心代码 StressTester.java

```
int clients = 50;
int requestsPerClient = 10;
ExecutorService pool = Executors.newFixedThreadPool(clients);

AtomicInteger success = new AtomicInteger();
AtomicInteger failed = new AtomicInteger();

long start = System.currentTimeMillis();

for (int i = 0; i < clients; i++) {
    pool.submit(() -> {
        for (int j = 0; j < requestsPerClient; j++) {
            try {
                Socket socket = new Socket("localhost", 8080);
                OutputStream out = socket.getOutputStream();
                InputStream in = socket.getInputStream();

                String request =
                    "GET / HTTP/1.1\r\n" +
                    "Host: localhost:8080\r\n" +
                    "Connection: close\r\n\r\n";

                out.write(request.getBytes());

                byte[] buffer = new byte[64];
                int len = in.read(buffer);
                String response = new String(buffer, 0, len);

                if (response.startsWith("HTTP/1.1 200")) {
                    success.incrementAndGet();
                } else {
                    failed.incrementAndGet();
                }

                socket.close();
            } catch (Exception e) {
```

```

        failed.incrementAndGet();
    }
}

});
}

pool.shutdown();
pool.awaitTermination(1, TimeUnit.MINUTES);

long end = System.currentTimeMillis();
int total = clients * requestsPerClient;
double seconds = (end - start) / 1000.0;

System.out.println("总请求数: " + total);
System.out.println("成功数: " + success.get());
System.out.println("失败数: " + failed.get());
System.out.println("吞吐量: " + total / seconds + " requests/s");

```

说明:

压力测试程序通过线程池模拟多个客户端同时访问服务器。每个线程循环发送多次 HTTP 请求，并根据服务器返回的状态码判断请求是否成功。最后统计总请求数、成功数、失败数、耗时和吞吐量，用于分析服务器并发处理能力。

2. 给出满足不同内容要求所进行测试的运行结果，包括客户端的运行结果及服务器端的运行结果，并简述所进行的测试。

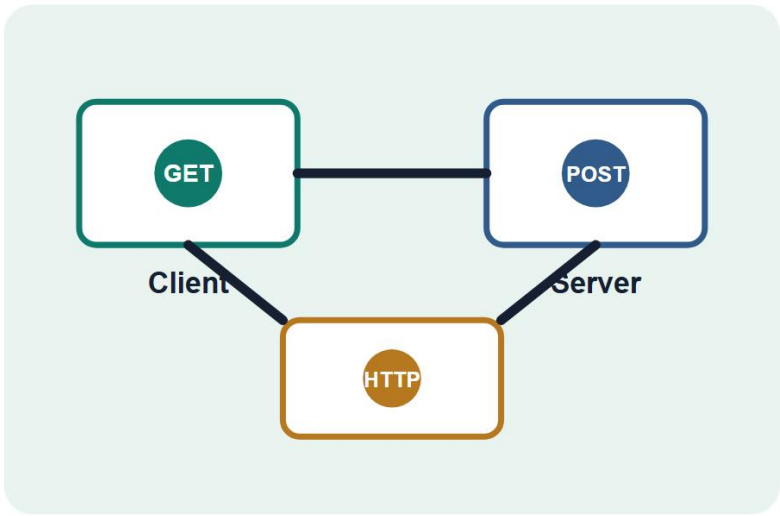
浏览器访问服务器首页，HTML、CSS、JS 正常加载



图：浏览器访问服务器首页，HTML、CSS、JS 正常加载

访问 `/images/network.svg` 可以看到服务器返回的图像资源，响应类型为 `image/svg+xml`，说明服务器能够返回文本之外的图片类型文件

浏览器访问 SVG 图片资源



图：浏览器访问 SVG 图片资源

客户端程序依次发送 GET /、HEAD /images/network.svg、POST /login、GET /api/session 请求。运行结果显示：GET 能获得 HTML 页面；HEAD 返回状态码和响应头但响应体长度为 0；POST 能保存用户名；最后一次 GET 能通过 Cookie 读取同一会话中的用户名和访问次数。

HTTP 客户端 GET、HEAD、POST 和 Cookie 会话测试输出

```
1  ===== GET / =====
2  HTTP/1.1 200 OK
3  Content-Type: text/html; charset=UTF-8
4  Cookie: LAB4_SESSION=4ff4eb6a-140a-475c-9ac3-7ea17901b03e
5  Body bytes: 1054
6  <!doctype html>
7  <html lang="zh-CN">
8  <head>
9    <meta charset="UTF-8">
10   <meta name="viewport" content="width=device-width, initial-scale=1.0">
11   <title>Lab 4 HTTP Socket Server</title>
12   <link rel="stylesheet" href="/style.css">
13 </head>
14 <body>
15   <main class="shell">
16     <section class="hero">
17       <div>
18         <p class="eyebrow">*****</p>
19         <h1>Java Socket HTTP Server</h1>
20         <p class="lead">***** Java HTTP ***** HTML CSS JS *****</p>
21       </div>
22       <img src="/image...
```

图：HTTP 客户端 GET、HEAD、POST 和 Cookie 会话测试输出

3.服务器性能分析总结。

对服务器进行压力测试，测试可支持多少个客户端同时访问，测试可支持多少个文件同时传输等，描述测试方法与测试结果。

压力测试使用 StressTester 程序完成。测试方法为：启动服务器后，分别模拟 50、100、200 个并发客户端，每个客户端连续发送 10 次 GET 请求，请求路径在首页、CSS、JS、SVG 图片和 JSON 接口之间轮换，以同时覆盖文本文件、脚本文件、样式文件和图像文件传输。

并发客户端数	每客户端请求数	总请求数	成功数	失败数	耗时	吞吐量
50	10	500	500	0	2.978 s	167.90 requests/s
100	10	1000	962	38	5.512 s	181.42 requests/s
200	10	2000	1924	76	10.716 s	186.64 requests/s

1	?50 ???
2	ZWNBSPClients: 50
3	Requests per client: 10
4	Total requests: 500
5	Success: 500
6	Failed: 0
7	Elapsed: 2.978 s
8	Throughput: 167.90 requests/s
9	
10	?100 ???
11	ZWNBSPClients: 100
12	Requests per client: 10
13	Total requests: 1000
14	Success: 962
15	Failed: 38
16	Elapsed: 5.512 s
17	Throughput: 181.42 requests/s
18	
19	?200 ???
20	ZWNBSPClients: 200
21	Requests per client: 10
22	Total requests: 2000
23	Success: 1924
24	Failed: 76
25	Elapsed: 10.716 s
26	Throughput: 186.64 requests/s

图：压力测试运行结果

从测试结果看，服务器在 50 个并发客户端、500 次总请求下全部成功，说明固定线程池能够稳定支持该规模的同时访问。并发提升到 100 和 200 时，请求总吞吐量提高到约 181.42 和 186.64 requests/s，但开始出现少量失败。失败主要可能来自短时间内大量 Socket 连接同时建立，服务器线程池大小为 32，连接排队和系统端口资源会带来压力。

文件同时传输方面，压力测试路径同时包含 HTML、CSS、JS、SVG 和 JSON 五类资源。50 并发时全部成功，可认为服务器能够稳定处理至少 50 个客户端同时访问并传输多种静态文件；更高并发下仍能完成大部分请求，但为了提高可靠性，可以增大线程池、设置连接队列长度、加入连接超时和更高效的文件缓存。

三、实验总结

本实验从零实现了一个简易 HTTP 服务器和客户端，对 HTTP 协议的请求行、

请求头、响应头、响应体以及状态码有了更直接的理解。相比直接使用 Web 框架，使用 Socket 编程需要手动处理字节流、Content-Length、文件类型判断和连接关闭等细节，因此更能体现 HTTP 协议在传输层之上的工作方式。

实验中遇到的主要问题包括：一是 POST 请求体不能只读取请求头，需要根据 Content-Length 再读取指定长度的请求体；二是 HEAD 请求虽然不返回响应体，但 Content-Length 仍应保持为目标资源长度；三是静态文件路径必须进行 normalize 检查，避免客户端通过 ../ 访问网站根目录之外的文件；四是 Cookie 的读取和 Set-Cookie 的返回需要保持名称一致，才能在多次请求间维持会话状态。

通过本实验，我掌握了 Java 服务器端 Socket 编程、HTTP 三种请求方法处理、静态网站部署、Cookie 会话状态管理和压力测试方法。后续如果继续改进，可以加入持久连接、更多 MIME 类型、访问日志文件、缓存机制和更完善的异常处理，使服务器更接近真实 Web 服务器。

指导教师批阅意见：	
成绩评定：	指导教师签字：周宇 年 月 日
备注：	

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

AI 使用说明

本人在完成本实验报告过程中，使用了大语言模型作为辅助工具。

使用工具名称：_____（如 ChatGPT / DeepSeek / Qwen 等）

使用用途：

- 查阅资料
- 代码检查
- 报告语言优化
- 其他...

本人承诺：

实验报告中的代码与内容均已理解，并能够向指导老师解释程序实现原理。

学生姓名：_____

学号：_____

日期：_____

提示词清单：

1. （例）帮我用 html 画一个关于 xxx 流程的图。输入：aa，中间量：bb，...
2. ...